Project Proposal: Cache-Aware Graph Reordering for Efficient Graph Neural Network Execution on GPUs

Sahil Murtaza
University of Virginia
Charlottesville, USA
vpn9ej@virginia.edu

Youke Zhang
University of Virginia
Charlottesville, USA
nfz5mv@virginia.edu

Zach Yu
University of Virginia
Charlottesville, USA
rqx6rs@virginia.edu

Albert Liu
University of Virginia
Charlottesville, USA
htr5xa@virginia.edu

Abstract

Graph Neural Networks (GNNs) suffer from significant hardware inefficiencies during training due to the irregular, sparse structure of real-world graphs. During neighbor aggregation, GPU threads must fetch feature vectors from non-contiguous memory locations, leading to poor spatial locality and high cache miss rates in the L2 cache and HBM subsystem. This proposal outlines a project to investigate, implement, and benchmark *cache-aware node reordering* as a lossless pre-processing step for GNN training. We formalize graph reordering as finding an optimal permutation π of the node index set that minimizes a matrix bandwidth or graph partitioning objective. We plan to implement and compare two concrete algorithms—Reverse Cuthill-McKee (RCM) and METIS-based community reordering—and measure their impact on GPU L2 cache hit rates and epoch training latency across standard Open Graph Benchmark datasets. Because reordering is a relabeling that commutes with all GNN operators, prediction accuracy is guaranteed to remain identical to the baseline.

Keywords

Graph Neural Networks, GPU Architecture, Cache Optimization, Node Reordering, Sparse Matrix Bandwidth, Graph Partitioning

1 Introduction: The Problem

The dominant bottleneck in accelerating Graph Neural Networks (GNNs) on GPUs is the *memory wall*. Modern GPU architectures are optimized for dense, contiguous general matrix multiplications (GEMMs), which exploit memory coalescing to saturate compute throughput. Real-world graphs, however, are highly sparse and topologically irregular, causing the core message-passing operations of GNNs to devolve into *scatter–gather* memory access patterns that defeat HBM coalescing and L2 cache reuse.

Message-Passing as Sparse Computation. Consider a GNN with L layers operating on an undirected graph $G = (V, E)$ with $n = |V|$ nodes and adjacency matrix $A \in \{0,1\}^{n\times n}$. Let $H^{(l)} \in \mathbb{R}^{n\times d_l}$ denote the matrix of node features at layer l, where d_l is the hidden dimension. The canonical message-passing update for node i at layer l is:

$$h_i^{(l+1)} = \sigma\Big(W^{(l)} \sum_{j\in\mathcal{N}(i)} c_{ij}\, h_j^{(l)}\Big), \tag{1}$$

where $W^{(l)} \in \mathbb{R}^{d_{l+1}\times d_l}$ is a learnable weight matrix, c_{ij} is a normalization coefficient (e.g. $c_{ij} = 1/\sqrt{\deg(i)\deg(j)}$ for GCN [?]), and σ is a non-linearity. Written over all nodes simultaneously, the aggregation step is a sparse matrix–dense matrix multiplication (SpMM):

$$\hat{H}^{(l)} = \tilde{A}\, H^{(l)}, \qquad \tilde{A} = D^{-1/2} A\, D^{-1/2}, \tag{2}$$

where $D = \mathrm{diag}(\deg(1), \ldots, \deg(n))$. In memory, $\tilde{A}$ is stored in Compressed Sparse Row (CSR) format. Row i of the CSR structure lists the column indices of the non-zero entries; these indices determine *which rows of* $H^{(l)}$ must be fetched during aggregation.

Why Random Orderings Cause Cache Misses. If the node indices are assigned arbitrarily (e.g. by order of ingestion), connected nodes can have indices that are far apart. In CSR storage, the column indices in row i will scatter across a wide range, causing the GPU to request rows of $H^{(l)}$ from distant memory addresses. Since the L2 cache on modern NVIDIA GPUs (e.g. 40 MB on A100) can only hold a small fraction of $H^{(l)}$ for large graphs, these distant accesses trigger *capacity and conflict misses*. Each cache miss forces a 128-byte sector fetch from HBM, stalling the streaming multiprocessors (SMs) and leaving compute units idle.

Project Goal. We aim to find a permutation $\pi : V \to V$ of the node index set such that, after relabeling every node i as $\pi(i)$, connected nodes tend to receive *nearby* indices. This makes the non-zero entries of $\tilde{A}$ cluster near the main diagonal, so the SpMM in Eq. (**??**) accesses rows of $H^{(l)}$ that are close in memory address space and thus more likely to reside in the same L2 cache line.

2 Mathematical Formulation of Graph Reordering

2.1 Permutations and the Reordered System

Let π be a permutation on $\{1, \ldots, n\}$ and let $P \in \{0,1\}^{n\times n}$ be the associated permutation matrix, defined by $P_{ij} = 1$ iff $\pi(i) = j$. Applying π to the graph yields the reordered adjacency matrix

$$A' = P\, A\, P^\top. \tag{3}$$

Because P is orthogonal ($P^\top P = I$), this is a similarity transform that preserves the spectrum of A and all structural invariants. The reordered feature matrix and degree matrix are

$$H'^{(l)} = P\, H^{(l)}, \qquad D' = P\, D\, P^\top. \tag{4}$$

Since the GNN update in Eq. (??) is *equivariant* to node relabeling, training on $(A', H'^{(0)})$ produces the same per-node outputs as training on $(A, H^{(0)})$ up to the same permutation, and thus the same classification or regression loss. Reordering is therefore a *lossless* transformation.

2.2 Bandwidth Minimization

The **bandwidth** of a matrix A under ordering π is

$$B_\pi(A) = \max_{(i,j)\in E} \left|\pi(i) - \pi(j)\right|. \tag{5}$$

A small bandwidth means every non-zero entry $A'_{\pi(i),\pi(j)}$ lies within $B_\pi(A)$ positions of the diagonal. During SpMM, this guarantees that the column indices in any CSR row span an interval of width at most $B_\pi(A)$, so the rows of $H'^{(l)}$ accessed by a single aggregation step lie within a contiguous memory region of size $B_\pi(A) \cdot d_l$ floats, improving cache line reuse.

Bandwidth minimization seeks

$$\pi^* = \arg\min_\pi \; B_\pi(A), \tag{6}$$

which is NP-hard in general [?]. In practice, heuristics such as Cuthill-McKee (CM) and its reversal (RCM) [?] provide near-optimal results for many graph families.

2.3 The Reverse Cuthill-McKee (RCM) Algorithm

The Cuthill-McKee algorithm is a breadth-first-search-based heuristic for bandwidth reduction. Given graph G:

(1) **Peripheral seed selection.** Choose a starting node s with small eccentricity (often found by a two-sweep heuristic: pick an arbitrary node, BFS to find the farthest node, then BFS again from that node).
(2) **Level-set traversal.** Initialize queue $Q = \{s\}$ and an empty ordering list R. While Q is non-empty:
   (a) Dequeue a node v from Q and append v to R.
   (b) Enqueue the unvisited neighbors of v in *ascending degree order*.
(3) **Reversal.** The Reverse Cuthill-McKee ordering is $\pi_{\text{RCM}}(i) = R[n-i]$ (i.e., reverse the list R).

The reversal step often further reduces bandwidth in practice. The overall complexity is $O(|E| \log |E|)$ due to the neighbor-sorting step. We will use the `scipy.sparse.csgraph.reverse_cuthill_mckee` implementation, which returns the permutation vector π directly.

2.4 The METIS Graph Partitioning Objective

METIS [?] partitions V into k disjoint subsets $V_1, \ldots, V_k$ by minimizing the **edge cut**:

$$\text{EdgeCut}(V_1, \ldots, V_k) = \left|\{(i,j) \in E : i \in V_p,\; j \in V_q,\; p \neq q\}\right|, \tag{7}$$

subject to a balance constraint $|V_p| \le (1+\epsilon)\, n/k$ for all p. Minimizing the edge cut maximizes the fraction of edges whose endpoints share the same partition. We construct the reordering permutation π_{METIS} by numbering all nodes in V_1 first (indices $1, \ldots, |V_1|$), then all nodes in V_2, and so on:

$$\pi_{\text{METIS}}(v) = \sum_{q=1}^{p-1} |V_q| + \text{rank}(v, V_p), \qquad v \in V_p, \tag{8}$$

where $\text{rank}(v, V_p)$ is the position of v within partition V_p (ordered arbitrarily, or recursively by a nested bisection). This produces a block-diagonal-like structure in A': intra-partition edges form dense diagonal blocks of size $\approx n/k$, while the minimized inter-partition edges appear as sparse off-diagonal entries. During SpMM, the GPU processes rows within a partition whose column indices fall predominantly within the same block, causing the required rows of $H'^{(l)}$ to reside in a contiguous $\approx (n/k) \cdot d_l$-float region that fits well in L2 cache.

The number of partitions k is a tunable hyperparameter: larger k yields smaller blocks (better L2 fit) but more inter-partition edges (more off-diagonal scatter). We will sweep $k \in \{4, 8, 16, 32, 64\}$ and report the Pareto trade-off between edge cut fraction and cache hit rate.

2.5 Comparison of Objectives

Table ?? summarizes the two reordering strategies.

Table 1: Comparison of the two reordering objectives.

Property	RCM	METIS
Objective	Min bandwidth B_π	Min edge cut
Structure of A'	Band-diagonal	Block-diagonal
Complexity	$O(\|E\| \log \|E\|)$	$O(\|E\|)$ amortized
Tuning	Seed selection	Partition count k
Best for	Low-diameter graphs	Community-rich graphs

3 Existing Literature

The intersection of graph layout optimization and deep learning systems is a highly active area. This project builds upon several foundational and contemporary works:

- **Cuthill & McKee (1969) [?]:** Introduced the BFS-based bandwidth reduction heuristic for sparse matrices; the reversed variant (RCM) remains a standard pre-processing step in finite-element solvers.
- **METIS (Karypis & Kumar, 1998) [?]:** Developed a multilevel k-way graph partitioning algorithm that coarsens the graph, partitions the coarsened version, and then uncoarsens with local refinement, achieving near-linear time.
- **Rabbit Order (Arai et al., 2016) [?]:** Introduced hierarchical community-based node reordering that maps graph topology to cache hierarchy levels.
- **GCN (Kipf & Welling, 2017) [?]:** Formalized the spectral-motivated convolutional layer that makes the SpMM in Eq. (??) the computational core of graph learning.
- **GNNAdvisor (Wang et al., 2021) [?]:** Introduced 2D workload management and lightweight node renumbering to improve spatial locality for GNN kernels on NVIDIA GPUs.
- **Bayer et al. (2024) [?]:** Provided an empirical study evaluating diverse reordering strategies on PyTorch Geometric, demonstrating that even lightweight algorithmic reordering can reduce GPU training time by up to 30%.

4 Methodology

4.1 Datasets

We will use medium-to-large-scale datasets from the Open Graph Benchmark (OGB) [?], selected to expose hardware memory bottlenecks at two different scales:

- **`ogbn-arxiv`** (169K nodes, 1.1M edges): A citation network of arXiv papers with 128-dimensional word2vec features. Small enough for rapid iteration but large enough to observe cache effects.
- **`ogbn-products`** (2.4M nodes, 61.8M edges): An Amazon co-purchasing network. At this scale, the feature matrix $H \in \mathbb{R}^{2.4\text{M}\times 100}$ occupies ~960 MB, substantially exceeding the 40 MB L2 cache on an A100 GPU.

Data will be acquired programmatically via the `torch_geometric.datasets` API.

4.2 GNN Architectures (Workload Models)

We benchmark reordering across two architectures that exercise different memory access patterns:

(1) **GraphSAGE [?]:** Samples a fixed-size neighborhood per node; the sampled column indices determine the memory footprint per aggregation. Reordering affects the spatial coherence of sampled indices.
(2) **Graph Attention Network (GAT) [?]:** Computes attention coefficients α_{ij} over all neighbors before aggregation, requiring *two* accesses to each neighbor's features (once for attention, once for weighted sum). This doubles the cache pressure per edge, amplifying the benefit of locality.

4.3 Reordering Pipeline

The offline pre-processing pipeline operates as follows:

(1) Load the graph $(A, H^{(0)}, y)$ from OGB.
(2) Compute the permutation vector π using either RCM (§**??**) or METIS (§**??**).
(3) Construct the permutation matrix P and apply Eq. (**??**): set $A' = PAP^\top$, $H'^{(0)} = PH^{(0)}$, and $y' = Py$.
(4) Rebuild the CSR/COO edge index for PyTorch Geometric and proceed with standard training.

Implementation will use `scipy.sparse.csgraph.reverse_cuthill_mckee` for RCM and the `pymetis` Python binding for METIS.

4.4 Hardware Profiling

To establish a causal link between reordering and performance, we will profile the compiled CUDA kernels using **NVIDIA Nsight Compute (`ncu`)**. Key metrics include:

- `l2_cache_hit_rate`: fraction of L2 sector requests serviced without going to HBM.
- `dram_read_throughput`: sustained HBM read bandwidth, which should *decrease* after reordering if fewer cache misses occur.
- `sm_occupancy`: warp occupancy, which may increase if fewer warps are stalled on memory.

5 Evaluation Plan

The project will be evaluated through a controlled ablation study comparing baseline graphs (original ordering) against each reordered variant.

- **Primary Systems Metric — Epoch Training Time.** Mean wall-clock time per epoch (ms/epoch), averaged over 50 epochs after a 10-epoch warmup. We target a 15–30% latency reduction, consistent with prior work [?].
- **Secondary Hardware Metric — L2 Cache Hit Rate.** Measured via `ncu` on the SpMM kernels. We expect reordered graphs to improve the L2 hit rate by 10–40 percentage points on `ogbn-products`.
- **ML Sanity Check — Node Classification Accuracy.** Because reordering is a similarity transform (Eq. (**??**)), accuracy must remain *identical* to the baseline within floating-point tolerance. Any deviation would indicate a bug in the pipeline, not a property of the method.

6 Project Timeline and Expected Outcomes

The project is organized into four sequential phases:

(1) **Environment and Baselines.** Set up the GPU profiling environment, acquire OGB datasets, implement baseline GraphSAGE and GAT models, and record initial timing and cache metrics.
(2) **Reordering Pipeline.** Implement the RCM and METIS reordering pipelines, validate correctness by checking that $A' = PAP^\top$ preserves the edge set, and verify that classification accuracy is unchanged.
(3) **Execution and Profiling.** Run reordered training at scale, sweep METIS's partition count k, collect `ncu` hardware traces, and build the ablation table.
(4) **Analysis and Reporting.** Visualize adjacency matrix sparsity patterns (before/after), plot the cache hit rate vs. training time Pareto frontier, and assemble the final report.

Expected Outcomes.

(1) A reusable, open-source PyTorch Geometric pipeline for automatic cache-aware graph reordering.
(2) A benchmarking report quantifying the relationship between matrix bandwidth (Eq. (**??**)), edge cut (Eq. (**??**)), GPU L2 cache hit rate, and GNN training latency.
(3) Guidelines for practitioners on *when* each reordering strategy is most beneficial (graph size, density, community structure).

Temporary page!

LaTeX was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because LaTeX now knows how many pages to expect for this document.